جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

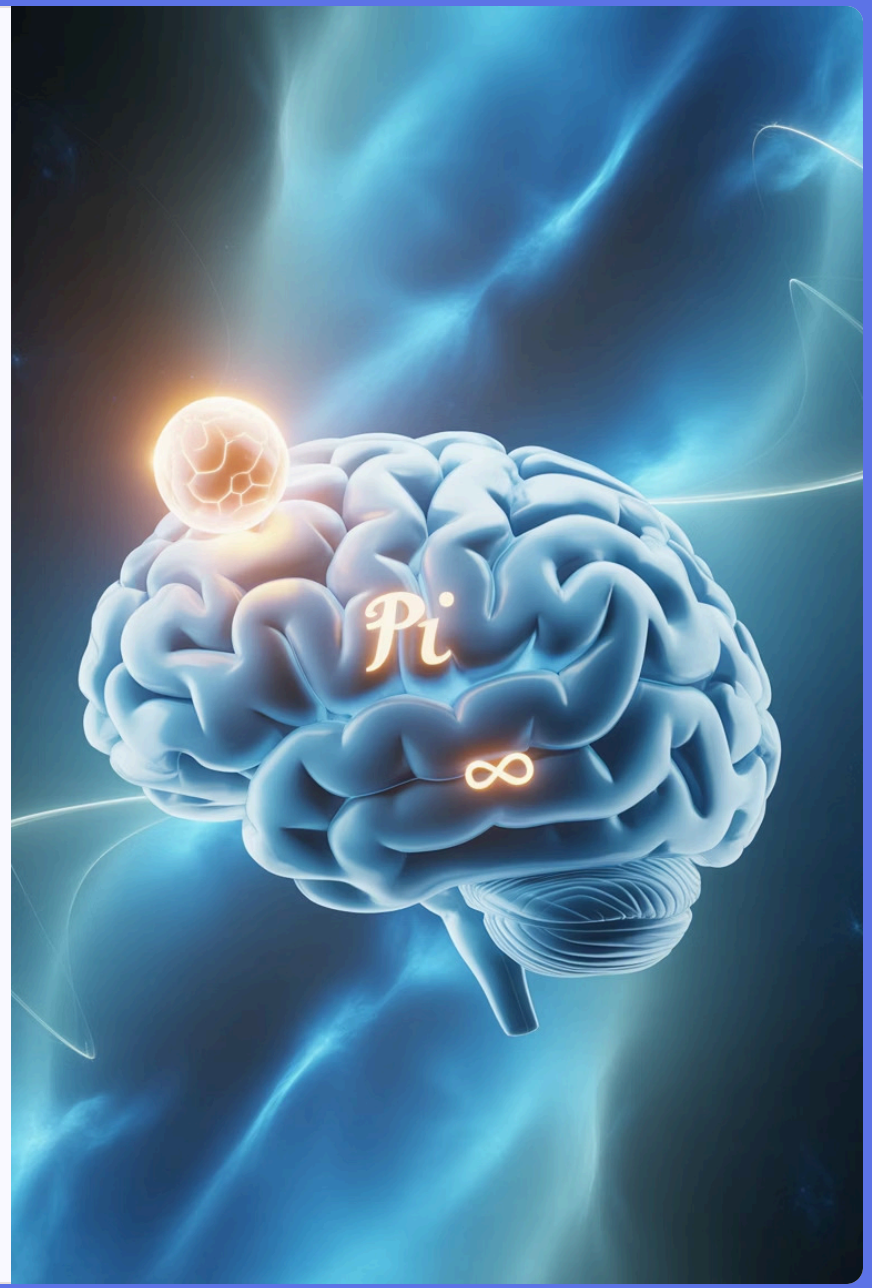
Introduction to Artificial Intelligence (SE 444)

Lecture 11: Bayesian Networks & Probabilistic Inference [Part 2]

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture introduces **Bayesian Networks** as powerful probabilistic graphical models, essential for managing uncertainty and complexity in AI. We explore their formal definition (DAGs + CPTs), key concepts like **conditional independence** and **d-separation**, and various inference methods from exact to approximate.





Exact Inference Problem

Given evidence $E = e$, compute posterior:

$$P(Q \mid E)$$

Where:

- **Q**: Query variables we want to know
- **E**: Evidence variables we observe
- **e**: Specific observed values

Exact Inference Problem

🎯 The Challenge of Belief Updating

When working with **Bayesian Networks**, the central task is to compute how our **beliefs** change when we observe new **evidence**.

Goal: Compute Posterior Probability

$$P(Q \mid E = e)$$

- **Q (Query Variables) 🤔**

  The variable(s) we want to reason about.

  Example:

  "Does the patient have the disease?"

- **E (Evidence Variables) 🔬**

  The variables we have observed.

  Example:

  Fever = True, Cough = True

- **e (Observed Values) ✅**

  The specific assignments to evidence variables.

  Example:

  E = {Fever = T, Cough = T}

Exact Inference Problem: Belief Updating

🔍 What Are We Really Doing?

We want to update our **belief** in **Q** after seeing evidence **e**.

- Before evidence: **prior belief**
- After evidence: **posterior belief** (what we want to compute)

💡 Intuition

> "Given what I know now, what is the probability that Q is true?"

Example: If the alarm goes off, what is $P(\text{Burglary} = \text{True} \mid \text{Alarm} = \text{True})$?

Inference by Enumeration Just the Chain Rule and Bayes' Rule Applied Naively?

Inference by Enumeration computes:

$$P(Q \mid E = e) = \frac{P(Q, e)}{P(e)}$$

To find $P(Q, e)$, it first expands the **full joint distribution** based on the network structure:

$$P(X_1, ..., X_n) = \prod_i P(X_i \mid Parents(X_i))$$

Then, it **sums out all hidden variables** (those not in Q or E) through brute-force marginalization:

$$P(Q, e) = \sum_h \prod_i P(x_i \mid parents(x_i))$$

This explicit computation and summing makes enumeration "naive" and computationally intensive, especially for large networks.

Inference by Enumeration (Conceptual Overview)

🧠 What Is Inference by Enumeration?

Inference by Enumeration is a method to compute the posterior probability:

$$P(Q \mid E = e)$$

by **summing over all possible worlds** consistent with the evidence. It's a foundational, though often computationally intensive, exact inference technique.

🔑 Core Idea: A Three-Step Process

1

Expand Full Joint Distribution

Utilize the Bayesian Network's factorization to write out the full joint distribution:

$$P(X_1, \ldots, X_n) = \prod_i P(X_i | \text{Parents}(X_i))$$

2

Enumerate Hidden Variables

Identify and list all combinations of values for the **hidden variables** ***(those not in the query Q or evidence E).***

3

Sum Probabilities

Sum the probabilities of all joint assignments that are consistent with the observed evidence and support the event you are querying.

Inference by Enumeration

The Central Question

You've built a Bayesian Network. Now what? **How do you use it to answer questions?**

$$P(\text{Query} \mid \text{Evidence}) = ?$$

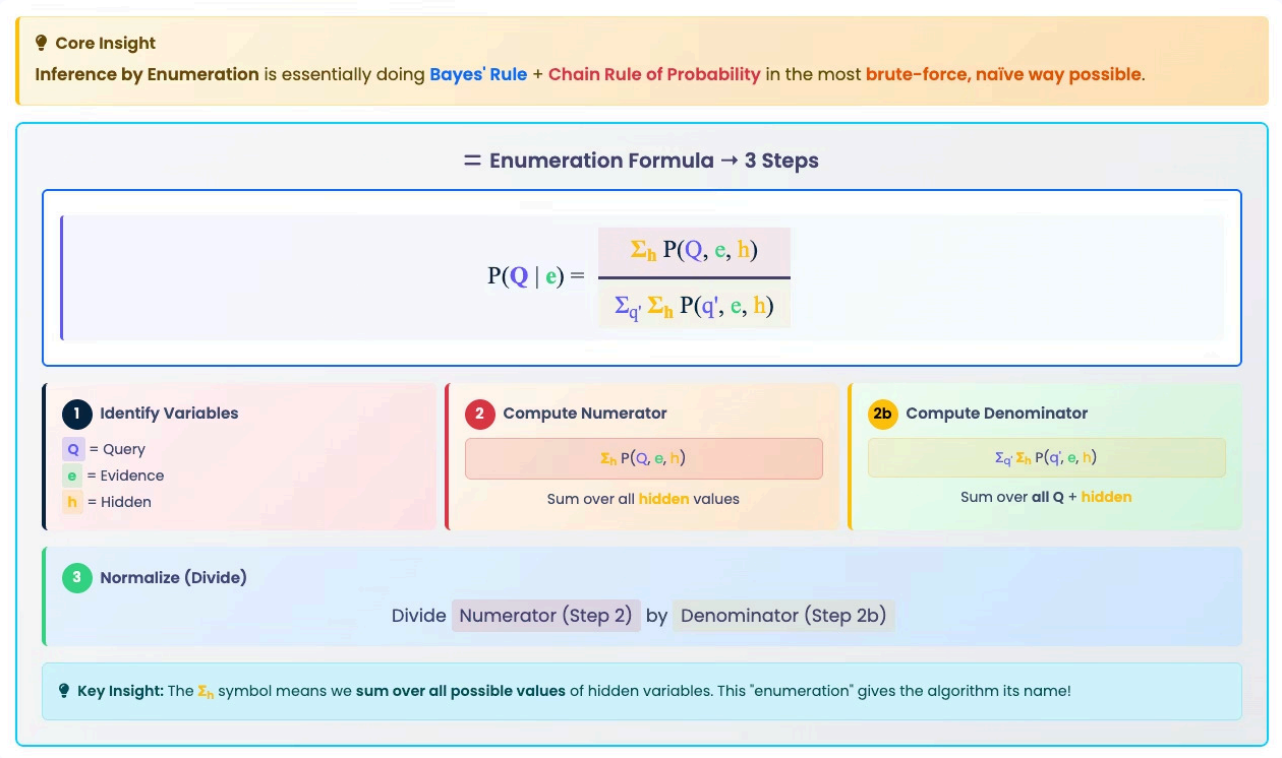
Inference is the process of computing these conditional probabilities from the BN.

Query	Evidence	Hidden
Burglary	JohnCalls, MaryCalls	Earthquake, Alarm
What we want to know	**What we observed**	**What we don't know**
"Is there a burglary?"	"Both neighbors called"	"Sum over all possibilities"

Goal of This Topic

Learn **Enumeration** — the most straightforward exact inference algorithm. It's conceptually simple but computationally expensive. Understanding it is crucial for learning faster algorithms later!

Enumeration Algorithm



Inference by Enumeration: Hidden Variables & Limitations

What Are Hidden Variables?

In the context of probabilistic inference, **hidden variables** are:

- Variables that are **not queried** (not Q)
- Variables that are **not observed** (not E)

These variables must be **summed out** to compute the desired posterior probability.

Practical Limitations

Computational Cost

Requires exploring every possible assignment to the hidden variables, leading to an **exponential time complexity** in their number.

Scalability Issues

Due to its high complexity, Inference by Enumeration is only practical for **very small Bayesian Networks** with a limited number of variables.

🌟The Best Intuition

VE = Cleaning Up the Calculation Before You Do It

Enumeration = doing a huge messy calculation as-is.

Variable Elimination = simplifying the expression before calculating.

Just like in algebra:

- **This is Enumeration:** $(a+b)(c+d)(e+f) \quad (\text{expand everything})$
- **This is VE:** Simplify first, eliminate unneeded variables, reuse results

VE is literally the "factor & simplify" trick from high school algebra, but applied to probabilities.

Variable Elimination (Concept & Intuition)

The Most Important Exact Inference Algorithm

Variable Elimination computes queries like:

$$P(Q \mid E = e)$$

more efficiently than Enumeration by reusing intermediate results instead of recomputing them repeatedly.

✨ Why VE Is More Efficient

- Avoids generating the full joint distribution
- Avoids summing over variables multiple times
- Uses **factor operations** to keep computations local
- Complexity depends on network structure (treewidth), not number of variables

💡 Key Idea

Instead of summing over **all hidden variables at once**, VE eliminates **one variable at a time**:

- Combine only the factors that involve that variable
- **Sum it out**
- Replace those factors with a **new smaller factor**
- Greatly reduces duplicate calculations

🧠 Intuition

Think of VE as doing inference by **carefully organizing the arithmetic** instead of brute-forcing it.

The Key Transformation



Enumeration vs. Variable Elimination

Side-by-Side Comparison

Enumeration

$$P(B,A) = P(A,B,C=T,D=T) + P(A,B,C=T,D=F) + P(A,B,C=F,D=T) + P(A,B,C=F,D=F)$$

Computation: 4 full joint probabilities
Redundancy: Computes P(A,B,C) twice!

Variable Elimination

$$P(B,A) = P(A,B,C=T) \times [P(D=T) + P(D=F)] + P(A,B,C=F) \times [P(D=T) + P(D=F)]$$

Computation: Eliminate D first ($\Sigma_D = 1$)
Efficiency: Computes P(A,B,C) **once**!

Variable Elimination Algorithm (Step-by-Step)

Goal

Compute $P(Q \mid e)$ efficiently by eliminating variables one at a time

1 Initialize Factors

What to do:

Create one factor for each CPT in the network

Example: For network $A \rightarrow B \rightarrow C$

- $f_A = P(A)$
- $f_B = P(B|A)$
- $f_C = P(C|B)$

Why:

Each factor represents a piece of the joint probability. We'll combine them strategically to avoid redundant computation.

Variable Elimination Algorithm (Step-by-Step)

🧩 VE Algorithm for Query

$$P(X \mid E = e)$$

2 Eliminate Hidden Variables (One at a Time)

What to do:

For each hidden variable X:

1. **Multiply** all factors that mention X
2. **Sum out** X from the product
3. Store the result as a new factor

Why:

By eliminating variables one at a time, we create smaller intermediate results that we can reuse, avoiding the exponential blowup of enumeration.

Mini Example: Eliminating C

Before:

$f_1(B,C)$

$f_2(C,D)$

After eliminating C:

$f_{new}(B,D) = \Sigma_C [f_1 \times f_2]$

Variable Elimination Algorithm (Step-by-Step)

3 Multiply & Normalize

What to do:

1. **Multiply** all remaining factors
2. **Normalize** to get probability distribution

$$P(Q|e) = \alpha \cdot [f_1 \times f_2 \times \ldots]$$

where α ensures sum = 1

Why:

After eliminating all hidden variables, we're left with factors only over the query and evidence. Normalizing gives us the final conditional probability.

The Key Pattern

Initialize → Eliminate X_1 → Eliminate X_2 → ... → Normalize

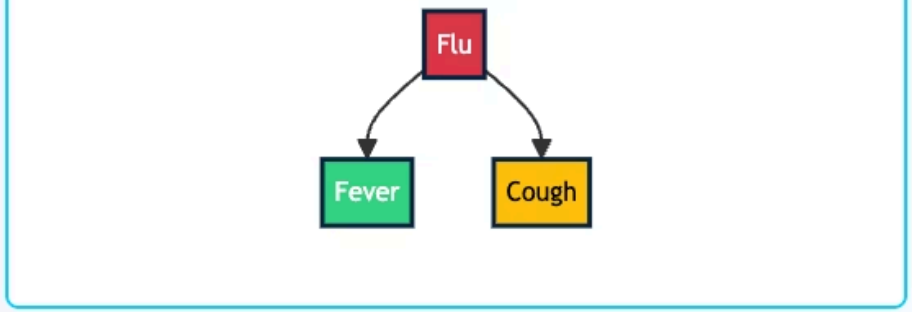
Each elimination step reduces the problem size!

Numerical Example

Query: $P(\text{Flu} \mid \text{Fever=Yes})$

A simple 3-node network: *Flu → Fever, Flu → Cough*

Network



Given CPTs

Flu	P
Yes	0.1
No	0.9

P(Fever|Flu)	
F=Y	0.8
F=N	0.2

P(Cough|Flu)	
F=Y then C=Y	0.7
F=N then C=Y	0.3

Variable Elimination Algorithm (Step 1and Step2)

Step 1: Initialize Factors & Set Evidence

Create factors and **instantiate evidence (Fever=Yes)**

$f_1 = P(Flu)$

Values:

$f_1(Flu=Yes) = 0.1$

$f_1(Flu=No) = 0.9$

$f_2 = P(Fever=Yes|Flu)$

Evidence set!

Values:

$f_2(Flu=Yes) = 0.8$

$f_2(Flu=No) = 0.2$

$f_3 = P(Cough|Flu)$

Values:

$f_3(Flu=Yes, Cough=Yes) = 0.7$

$f_3(Flu=No, Cough=Yes) = 0.3$

Note: We set Fever=Yes in factor f_2 so it becomes a function only of Flu

Step 2: Eliminate Cough (Hidden Variable)

Only f_3 contains Cough, so we sum it out:

$$f_4(Flu) = \Sigma_{Cough} [f_3(Cough, Flu)]$$

Result: $f_4(Flu) = 1$ for all Flu values (probabilities sum to 1)

Variable Elimination Algorithm (Step 3)

Step 3: Multiply Remaining Factors

Multiply all factors over Flu:

$$P(\text{Flu}, \text{Fever}=\text{Yes}) = f_1(\text{Flu}) \times f_2(\text{Flu}) \times f_4(\text{Flu})$$

Flu=Yes:

$$P(\text{Flu}) \times P(\text{Fever}=\text{Yes}|\text{Flu}=\text{Yes}) \times 1$$
$$= 0.1 \times 0.8 \times 1 = 0.08$$

Flu=No:

$$P(\text{Flu}) \times P(\text{Fever}=\text{Yes}|\text{Flu}=\text{No}) \times 1$$
$$= 0.9 \times 0.2 \times 1 = 0.18$$

Step 4: Normalize

P(Flu=Yes | Fever=Yes) = 0.08 / (0.08 + 0.18) =

0.308 (30.8%)